



DOCUMENTATION TECHNIQUE

Historique des versions

Version	Par qui ?	Quand ?	Quoi ?	Pourquoi ?
1.0	Alexia Legrand	20/02/24	Création du document	Comprendre comment fonctionne la documentation avec Dart
2.0	Jalil Jaajoui	05/03/24	Application du Template	Homogénéiser les documents selon Charte Graphique
3.0	Automne Boumaza	17/06/24	Fusion des documentations technique	Compléter la documentation avec le back



Sommaire

1) Fonctionnement	4
2) Node.js pour le Backend	5
2.1) Performance et Efficacité :	5
2.2) Non-bloquant et Asynchrone :	5
2.3) Large Écosystème :	5
2.4) Flexibilité :	6
2.5) Scalabilité :	6
3) PostgreSQL pour la Base de Données	6
3.1) Robustesse et Fiabilité :	6
3.2) Extensibilité et Fonctionnalités Avancées :	6
3.3) Sécurité :	7
3.4) Support des Requêtes Complexes :	7
3.5) Communauté Active et Support :	7
3) Architecture Dev ops	8
3.1) Contrôle de Version	8
3.2) CI/CD : GitHub Actions	8
3.3) Système de base de données	8
3.4) Schéma d'architecture	9



1) Fonctionnement

- La documentation sur dart est très simple, il suffit de commenter avec '///' une fonction, une class, ... pour créer de la doc.
- Il est aussi possible de définir les variables ou des méthodes avec [ma_variable].
- Enfin, il suffit d'exécuter "dart doc" pour générer la documentation.

poc package	
poc package LIBRARIES AddComponentPage BottomBarButtonCompo... BottomBarComponent createFloatingButtons CreatePage Editor EditorPage game1 game2 main ObjectComponent ObjectEditorPage PlayFloatingButtons playFloatingButtons PlayPage PlockComponent plockComponentsList ProfilePage RectanglePlockCompon... SearchPage TextPlockComponent	poc plock poc Getting Started This project is a starting point for a Flutter application. A few resources to get you started if this is your first Flutter project: • Lab: Write your first Flutter app • Cookbook: Useful Flutter samples For help getting started with Flutter development, view the online documentation, which offers tutorials, samples, guidance on mobile development, and a full API reference. Libraries AddComponentPage BottomBarButtonComponent BottomBarComponent createFloatingButtons CreatePage Editor EditorPage game1 game2

Si aucune documentation n'est spécifié pour une class, fonction, ... dart générera de la doc par défaut.

2) Node.js pour le Backend

2.1) Performance et Efficacité :

Node.js permet d'exécuter du code JavaScript à une vitesse élevée. Cette rapidité est cruciale pour traiter un grand nombre de requêtes simultanées, ce qui sera globalement le cas de notre réseau social Plock.

2.2) Non-bloquant et Asynchrone :

Grâce à son modèle d'entrées-sorties non bloquant et asynchrone, Node.js permet de gérer plusieurs opérations en même temps sans attendre la fin de chacune, ce qui sera d'une grande utilité dans le cas de notre application Plock qui pourrait devoir gérer un grand nombre de requêtes simultanément.

2.3) Large Écosystème :

Node.js bénéficie d'un énorme écosystème grâce au Node Package Manager, offrant une vaste gamme de modules et de librairies. Cela facilite et accélère le développement.

2.4) Flexibilité :

Node.js ne restreint pas les développeurs à un ensemble spécifique de règles ou de structures, leur permettant ainsi de construire l'application selon leurs besoins et préférences.

2.5) Scalabilité :

Node.js est conçu pour être scalable, facilitant la gestion des montées en charge de l'application.

3) PostgreSQL pour la Base de Données

3.1) Robustesse et Fiabilité :

PostgreSQL est reconnu pour sa robustesse, sa fiabilité et sa conformité aux standards SQL. Il offre des fonctionnalités avancées comme les transactions ACID (Atomicité, Cohérence, Isolation, Durabilité), essentielles pour garantir l'intégrité des données.

3.2) Extensibilité et Fonctionnalités Avancées :

PostgreSQL supporte un large éventail de types de données et permet l'extension de ses fonctionnalités grâce à des procédures stockées, des triggers, et des vues matérialisées. Cela le rend idéal pour des applications nécessitant des traitements complexes des données.

3.3) Sécurité :

Il offre des fonctionnalités de sécurité avancées, comme l'authentification forte, le contrôle d'accès granulaire et le chiffrement SSL, qui sont essentiels pour protéger les données sensibles.

3.4) Support des Requêtes Complexes :

PostgreSQL est particulièrement performant pour les requêtes complexes et le traitement de grandes bases de données, ce qui semblerait approprié au développement d'un réseau social visant à s'agrandir.

3.5) Communauté Active et Support :

Il bénéficie d'une communauté active et d'un bon support, ainsi que d'une documentation complète, assurant un suivi constant de l'évolution et un dépannage efficace.

3) Architecture Dev ops

3.1) Contrôle de Version

Utiliser Git avec GitHub pour le contrôle de version et le stockage du code, soutenant la collaboration et la révision de code à travers les pull requests.

→ Organisation Plock, répertoire déjà existant et ayant déjà les membres dessus.

3.2) CI/CD : GitHub Actions

pour l'intégration et la livraison continues, automatisant les tests, les constructions, et les déploiements vers les serveurs de test et de production.

→ Large panel de documentation sur GitHub Actions (Logique d'utiliser ça avec Github)

3.3) Système de base de données

R2 est une excellente option pour ce projet en raison de sa capacité à gérer des bases de données relationnelles à grande échelle avec une facilité de mise à l'échelle verticale et horizontale.

Son architecture sans serveur garantit une haute disponibilité et une durabilité des données, ce qui est crucial pour les applications nécessitant une accessibilité fiable.

Utilisation Effective Dart comme “lead tech”. Utilisation de ce guide comme référence afin d’écrire un code propre et suivant les bonnes pratiques du langage.

Google Cloud Storage peut offrir une intégration plus naturelle et des optimisations pour les projets Flutter, étant donné que Flutter est une technologie Google.

3.4) Schéma d’architecture

